

Vectorless RAG

RAG stands for Retrieval-Augmented Generation

<https://www.youtube.com/watch?v=f3zHina9MTo>

<https://medium.com/@visrow/what-is-pageindex-how-to-build-a-vectorless-rag-system-no-embeddings-no-vector-db-dc097fae3071>

Before jumping into that, let's first understand:

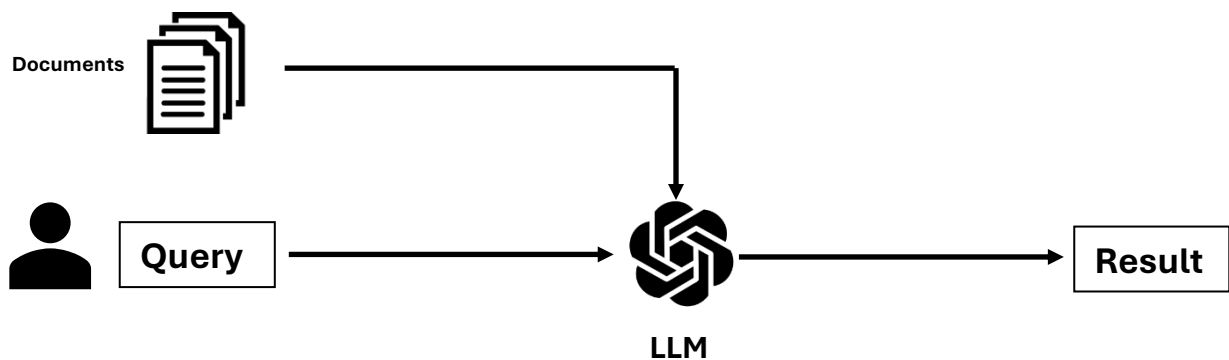
- What is **Traditional RAG**
- What is **Vector RAG**
- What problems exist in Vector RAG
- And how **Vectorless RAG (Page Index)** solves those problems using **reasoning models**

Problem Statement:

You have a lot of documents (like PDFs), and a user wants to ask questions on them using AI.

Naive Approach:

- Take all documents
- Put them into an LLM along with the user query
- Generate an answer

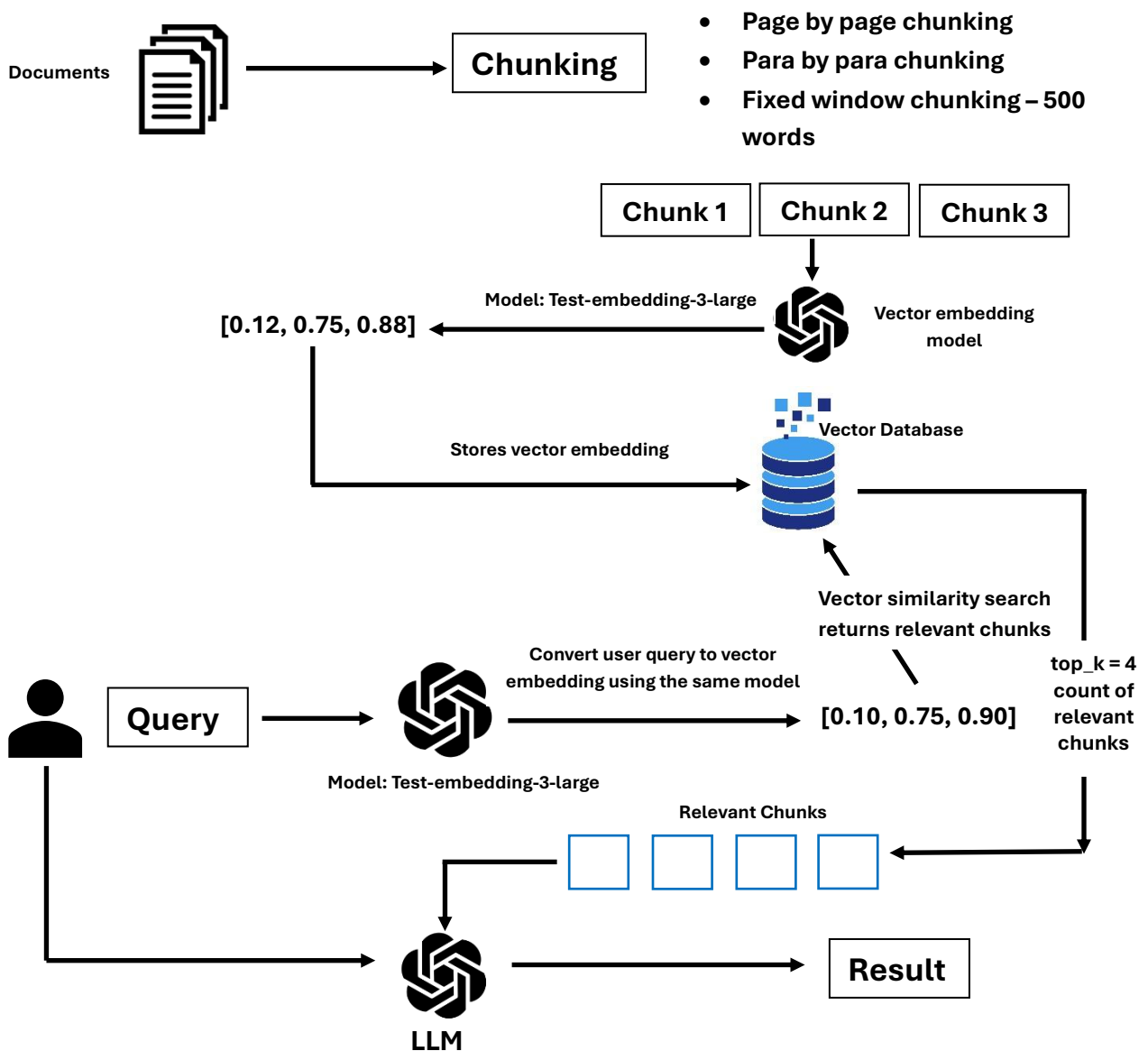


❌ Problems:

1. **Context Limit**
 - LLMs have limited context window

- You can't input 1000+ pages
2. **Hallucination**
- Too much data → model loses focus → generic answers
 - Output quality is not good
3. **Cost**
- Tokens = money
 - Sending full documents every time is expensive

Vector-based RAG



To solve this, we use **RAG pipeline with 2 phases:**

1 Indexing Phase

Steps:

1. Chunking

- Split document into small parts
- Example:
 - Page-based
 - Paragraph-based
 - Fixed-size (e.g., 500 words)

2. Embedding

- Convert each chunk into vectors (numbers)

3. Store in Vector DB

- Examples:
 - Pinecone
 - Chroma
 - Weaviate
 - Qdrant

2 Query Phase

Steps:

1. User asks a query
2. Convert query → vector embedding
3. Perform **vector similarity search**
4. Retrieve **top-K relevant chunks**
5. Send those chunks + query to LLM
6. Generate final answer

Most Retrieval-Augmented Generation (RAG) systems rely on embeddings and vector databases. They split documents into chunks, convert them into vectors, and retrieve answers using cosine similarity.

✗ Problems:

● 1 Chunking Problem (Biggest Issue)

✗ What happens:

- Documents are split into fixed chunks (e.g., 500 tokens)
- But meaning doesn't follow fixed sizes

⚠️ Example:

Chunk 1 → "The contract states that if the user violates..."

Chunk 2 → "...terms, the penalty will be applied as per clause 7"

👉 Meaning is broken across chunks

🚨 Impact:

- Model gets incomplete context
- Wrong or partial answers

🔴 2 Loss of Semantic Structure

✗ Problem:

Documents have structure:

- Sections
- Headings
- Relationships

👉 Vector RAG flattens everything into chunks

🚨 Impact:

- Model doesn't understand:
 - hierarchy
 - importance
 - relationships

🔴 3 Poor Handling of Cross-References

✗ Example:

- Page 5 says: "Refer to clause 10.2"
- Clause 10.2 is on Page 120

👉 Vector search may retrieve only one

🚨 Impact:

- Missing context → incorrect answer

🔴 4 Over-Reliance on Query Quality

✖ Problem:

- Retrieval depends on user wording

⚠ Example:

- Doc says: "Automobile"
- User asks: "Car"

👉 Embedding may fail to match correctly

🔔 Impact:

- Wrong chunks retrieved
- Garbage in → garbage out

🔴 5 Similarity ≠ Relevance

✖ Problem:

- Vector DB returns mathematically similar vectors
- Not always logically relevant

⚠ Example:

- Query: "Why did system fail?"
- Retrieved:
 - "System architecture overview" (similar words)
 - Not the actual failure reason

🔴 6 No Reasoning in Retrieval

✖ Problem:

- Retrieval = distance calculation (cosine similarity)

👉 No understanding like:

- cause-effect
- timeline
- logic

🔔 Impact:

- Can't answer complex queries properly

🔴 7 Fixed Top-K Retrieval Limitation

✖ Problem:

- You fetch top 3 / 5 / 10 chunks

👉 But:

- Important info may be in chunk #11

🔔 Impact:

- Missing critical data

8 Embedding Model Limitations

✗ Problem:

- Quality depends on embedding model

👉 Issues:

- Domain mismatch
- Outdated embeddings
- Language ambiguity

9 Cost & Storage Overhead

✗ Problem:

- Every chunk → embedding → storage

👉 Large docs = millions of vectors

🚨 Impact:

- High infra cost
- Scaling issues

10 Update / Re-indexing Problem

✗ Problem:

- If document changes:
 - Re-chunk
 - Re-embed
 - Re-store

🚨 Impact:

- Expensive + slow updates

1 1 Context Window Still Limited

Even after retrieval:

👉 You still send only:

- Top-K chunks

🚨 Impact:

- Important context might still be missing

1 2 Hallucination Still Possible

✗ Why:

- Retrieved chunks may be:
 - irrelevant
 - incomplete

👉 LLM fills gaps → hallucination

🧠 Simple Summary

Vector RAG Problems =
Chunking + Similarity + No Reasoning

Vectorless Rag or Page Indexing

PageIndex is a vectorless RAG architecture that retrieves information by reasoning over document structure instead of performing semantic search. Rather than treating a document as a flat pile of text, it treats it as a structured hierarchy — like a textbook with a table of contents.

PageIndex is a vectorless, reasoning-based Retrieval-Augmented Generation (RAG) approach that retrieves answers from long documents without using embeddings, chunking, or a vector database.

Instead of relying on semantic similarity search, PageIndex builds a **hierarchical Table of Contents (TOC) tree** from a document and uses a Large Language Model (LLM) to reason over that structure. The model first identifies the most relevant section using the document's hierarchy, then navigates to that section to generate a precise, cited answer.

Traditional RAG retrieves by similarity.

PageIndex retrieves by reasoning over structure.

This makes it particularly effective for structured, long-form content such as financial reports, legal contracts, regulatory filings, policy documents, and academic papers.

Why PageIndex Works?

PageIndex works because it separates two cognitive tasks:

1. **Navigation** — Determine where the answer should exist.
2. **Extraction** — Read only that section and generate the answer.

This mirrors how humans read:

When you want to know why something happened in a novel, you don't skim every page randomly.

You go to the chapter where the relevant event occurred.

PageIndex forces the LLM to behave the same way.

Vector RAG:

Query → Find similar chunks

Vectorless RAG:

Query → Traverse tree → Reason → Select path

How Query Flows in This Tree

Example Query:

👉 *"Why did revenue drop in Q3?"*

Step-by-step traversal:

1. Start at ROOT



2. LLM reads summaries of:

- Section A
- Section B
- Section C

→ Selects: Section B (most relevant)



3. Go deeper into Section B

- B1
- B2
- B3

→ Selects: B2 (Revenue Analysis)



4. Go deeper:

- B2.1

- B2.2

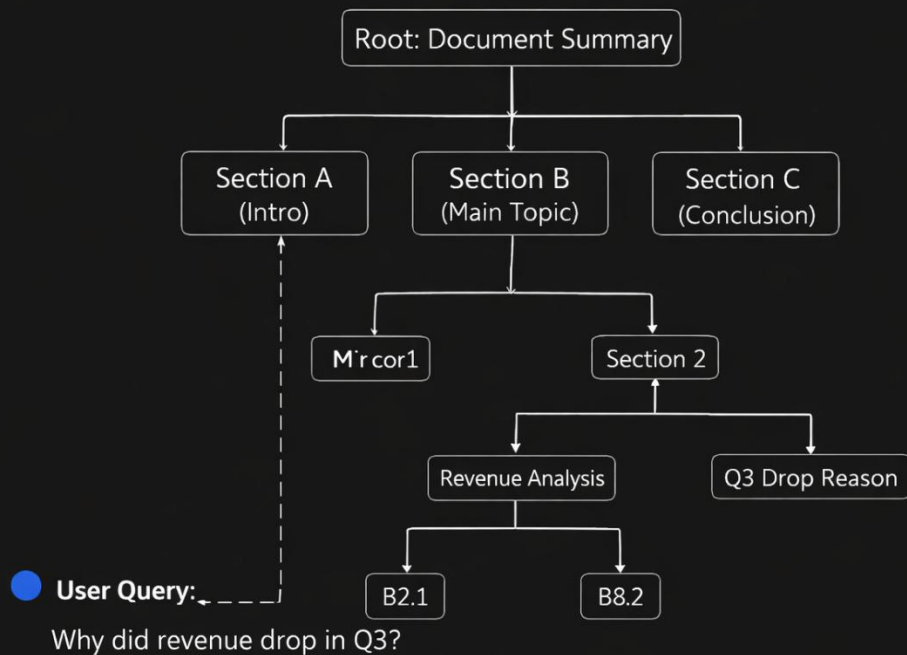
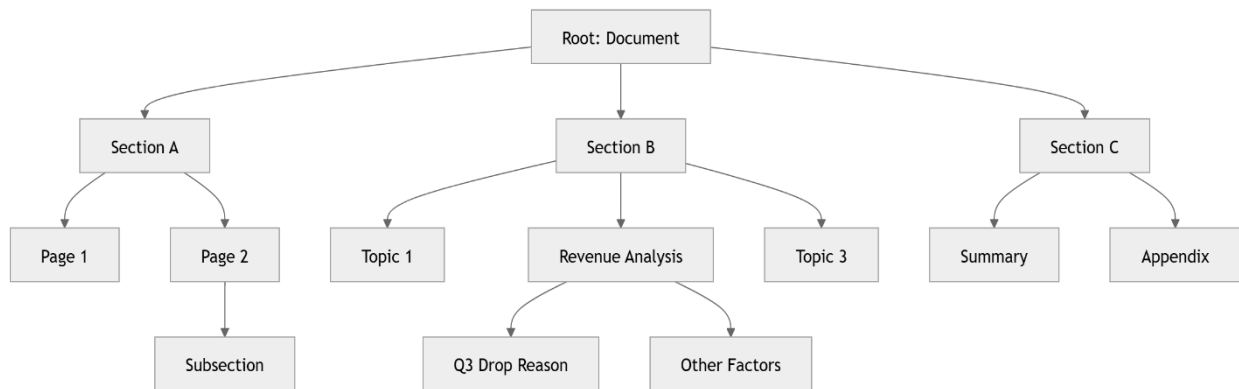
→ Selects: B2.1 (Q3 drop reason)

↓

5. Fetch original content (pages)

↓

6. Generate final answer



Vectorless Search

✖ Node Structure (What each circle contains)

Each node in tree:

Node:

- Title: "Revenue Analysis"
- Summary: "Explains Q3 drop"
- NodeID: points to actual document
- Children: [sub-nodes]

⚙️ ◆ Node Schema (Important for Implementation)

```
{
  "node_id": "sec_2_1",
  "title": "Revenue Drop Analysis",
  "summary": "Explains reasons for Q3 revenue decline",
  "children": [],
  "source_ref": {
    "page_start": 45,
    "page_end": 52
  }
}
```

🔄 ◆ Step-by-Step Pipeline

● Indexing Phase (Offline)

Documents

↓

LLM parses structure

↓

Extract:

- Sections
- Events
- Topics

↓

Build Tree (TOC Index)

↓

Store Tree (JSON / DB)

● Query Phase (Online)

User Query



LLM reasons over Tree



Find relevant nodes



Fetch original content



LLM generates answer

Why This Works Better

- ✓ Keeps full context
 - No chunk splitting
- ✓ Handles relationships
 - Parent ↔ Child links
- ✓ Multi-hop reasoning
 - ROOT → Section → Subsection
- ✓ Human-like behavior
 - Like reading index of a book